



Mechatronics & Embedded Deployment

Rhys Robbins

Department of Electrical and Computer Engineering, Lafayette College, Easton, PA.

Prof. Dr. Priyank Kalgaonkar

August 14, 2026

1. Abstract

Autonomous vehicle control software testing requires a safe, reliable and repeatable platform for control algorithms. In this project a skid steer four wheel drive rover was designed and built using off the shelf components an Arduino Mega and a Jetson Nano to implement total system functionality as an autonomous rover platform. This rover was designed and built with a custom power distribution system that incorporates individual 30A snap action breakers for each motor, as well as a main 120A breaker for the entire system. A dedicated relay based emergency stop (e-stop) circuit was also designed to disable motor power while maintaining control and compute system power facilitating system testing and debugging.

The rover features two primary control programs, a C/C++ program running on the Arduino Mega which acts as the low-level PWM driver for the motor controllers and a high-level Python program for the Jetson which acts as the control program for the entire rover. The Python program is designed to interface with a FS-IA6B 2.4GHz receiver paired with an FS-I6S to process wireless control inputs and transmit serial control commands to the Arduino Mega. This architecture enables full manual control over the rover while also acting as the framework for autonomous control programs that will be implemented later on.

System testing proved full functionality of the rover's power, safety, communication and motor control systems, establishing the platform as a safe, reliable and repeatable platform for future autonomous development.

2.Introduction

In the modern era Machine Learning (ML) is becoming one of the most prominent and impactful technologies on the planet. Thousands of companies throughout the world are developing ML algorithms that can be used for countless applications. Yet by nature this technology is unstable and nearly unpredictable in certain cases. This leads to the most important phase of developing ML, testing. Testing becomes even more important when considering something such as self operating vehicles as in most of these cases human lives could be at risk if things do not go as intended. For the purposes of this research only the fundamental training of ML algorithms is all that needs to be considered.

Development of a working autonomous vehicle is done using progressive testing methods. To simplify for the purpose of this project this can be thought about in five phases: Phase one is the design and creation of the ML algorithm with a physical platform that it is being built off of. This platform is likely to change slightly but its core functionalities will always be the same. For example if a self-driving car is being developed the exact chassis, engine, gears, horsepower, ect does not need to be decided during the algorithm creation it can be finalized much later.

Once this algorithm is designed, development enters phase two where it will go through countless simulated test runs where the algorithm learns how it should act. While this sounds like it may be enough to consider the algorithm complete, sim-to-real transfer or for the purpose of this paper, phase three is necessary for unconsidered factors in simulations. Simulations can only be so accurate and as a result all simulation models suffer from something called “reality gap”. The reality gap can significantly grow from many factors including but not limited to, sensor

noise, motor behavior, wheel slippage, communication latency, and environmental variation.

As a result this creates the need for a physical testbed that can fill in the reality gap created from the simulation after more testing and calibration take place. This testing starts out very simple and then once again is progressively made more difficult to ensure safe functionality of the algorithm. Finally, while outside the scope of this project phase four is where more complex environment testing takes place in the real world after deployment.

The rover developed in this project was designed to fill this role as a testbed for ML development mostly for phase three sim-to-real transfer non critical scenarios. As previously mentioned when working with ML algorithms programs can act in many unpredictable ways which is why there are many specific requirements for a capable ML testbed. These requirements were determined in collaboration with Dr. Priyank Kalgaonkar of Lafayette College's Electrical and Computer Engineering department and are as follows: The platform must include a reliable control software with the ability for many tests to be run in succession. A modular mounting mechanism for any sensors must be implemented for ease of testing and futureproofing. A powerful computer that can run complex ML algorithms in real time on the testbed itself. Finally most importantly safety features must be implemented throughout the system that allow for precise control of the systems onboard.

The rover was created to fulfill all of these requirements and has demonstrated them successfully through prototyping and testing. The final result utilizes a four wheel drive skid steer configuration and a modular control architecture that separates high level computing from low level motor control. This allows for future ML algorithms to be seamlessly implemented and evaluated without requiring any significant changes or modifications to the fundamental rover control system. The rover was also designed with dedicated power distribution boards, multiple

phases of circuit protection and an e-stop system providing a reliable, repeatable and safe testbed. The scope of this project is limited to the design, construction, and validation of the physical platform and its manual control architecture. The development of an autonomous software has been spearheaded by fellow EXCEL researcher Samuel Jackson. Its implementation, testing and calibration are outside of the scope of this project and will be picked up at a later date by Matthew Olins.

3.System Requirements

This system is intended to be used as an autonomous vehicle testbed for ML and control-system development, so several primary system requirements were established before the rover architecture was finalized. These requirements are summarized below.

Requirement	Design intent
Reliable control system	Support repeated manual and future autonomous tests without requiring significant low-level hardware changes. Manual control must also allow the rover to be driven and disabled before autonomous software is implemented.
Onboard computing power	Provide sufficient onboard computation for future perception, control, and machine-learning workloads in real time.
Modular mechanical and electrical platform	Allow sensors, computing hardware, and other electronics to be added or replaced without redesigning the full chassis.
Layered safety systems	Provide a physical drivetrain disconnect, a remote arming function, software timeouts, output limiting, and neutral-on-fault behavior.

Table 1: Primary system requirements.

4. System Design and Architecture

4.1. Overall System Architecture

The design of the rover began with two primary hardware constraints: the AndyMark Rover 8 Pneumatic chassis and a Jetson Nano P3450 for onboard computing. These were the only two major components predetermined before the rover design process began. Other component choices were primarily limited by cost, availability, compatibility, and the previously established system requirements.

The final rover architecture can be separated into several interacting subsystems. The mechanical platform and drivetrain provide physical movement, while individual motor controllers provide electrical control of each DC motor. An Arduino Mega operates as the low-level controller responsible for generating the PWM signals required by the motor controllers. A Jetson Nano acts as the high-level computing platform and processes commands from the wireless receiver before transmitting motor commands to the Arduino through serial communication.

The electrical architecture is separated into drivetrain power and control-system power. This separation allows the drivetrain to be disconnected through a dedicated emergency-stop circuit while allowing the Jetson Nano, Arduino Mega, and other control electronics to remain operational. This design is particularly important during autonomous software testing because it allows physical movement to be stopped without removing access to the computer responsible for running and debugging the software.

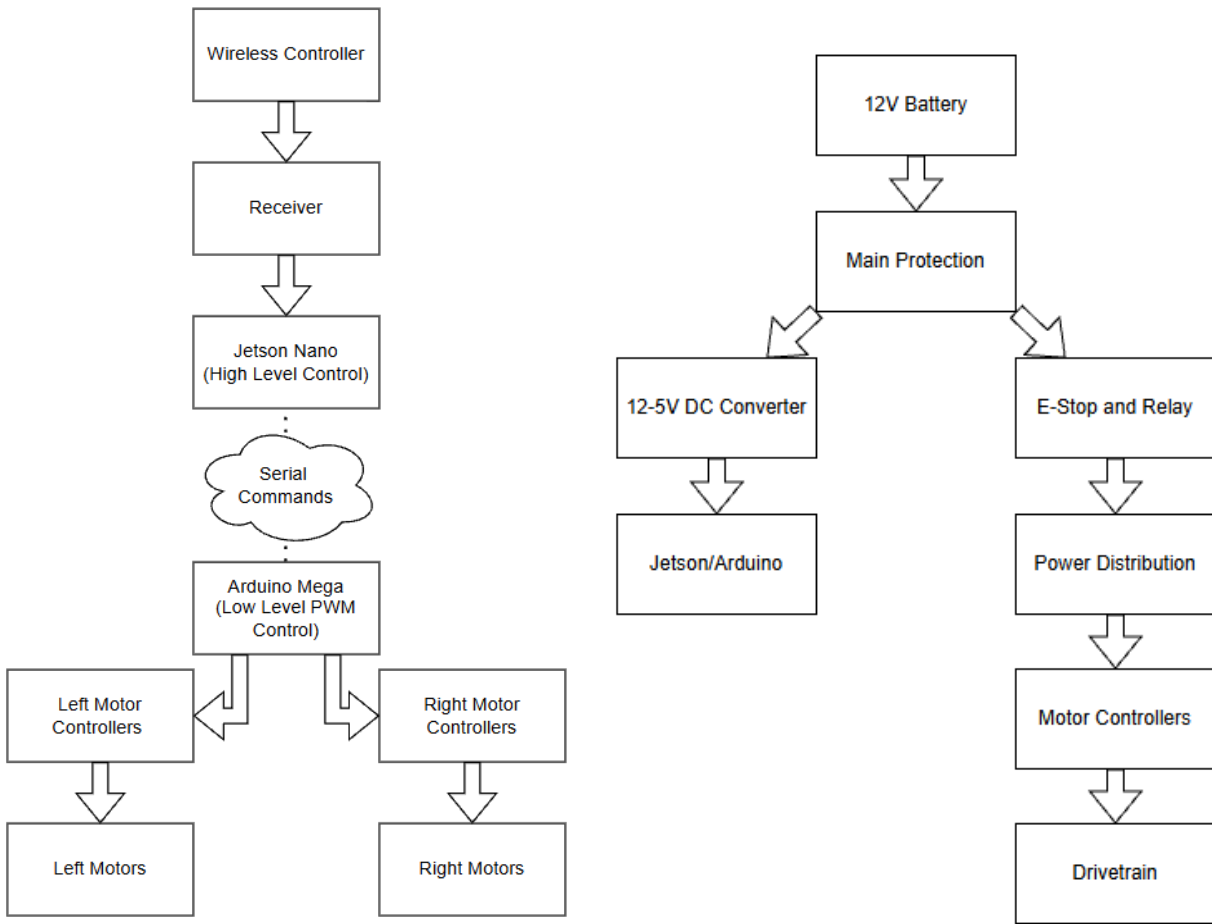


Figure 1: Overall control communication and power architecture.

4.2.Mechanical Design and Integration

4.2.1.Mechanical Design Requirements

Although the AndyMark Rover 8 Pneumatic chassis provided the primary mechanical platform and drivetrain for the rover it was only designed to mount the 120A breaker. Therefore a significant portion of mechanical design required determining the layout that would not only fit all the parts needed but also that would allow for cable management, proper cooling for the Jetson and serviceability. After this custom mounting hardware had to be designed to securely mount all critical components.

Space inside the chassis would be a major limitation as the battery, Jetson Nano, Arduino Mega, DC to DC converter, e-stop button, motor controllers, power distribution board, bus bars, and all wiring had to fit within the chassis. Not only that but because the plexiglass bottom is weak and was not designed to hold the selected hardware it would bend significantly under just the weight of all the components meaning additional support and alternative mounting solutions were needed.

For all these reasons individual custom mounting hardware had to be designed and 3D printed for almost every part minus the 120A breaker, the already mounted gearboxes/motors and the relay which used an existing mounting hole in the plexiglass.

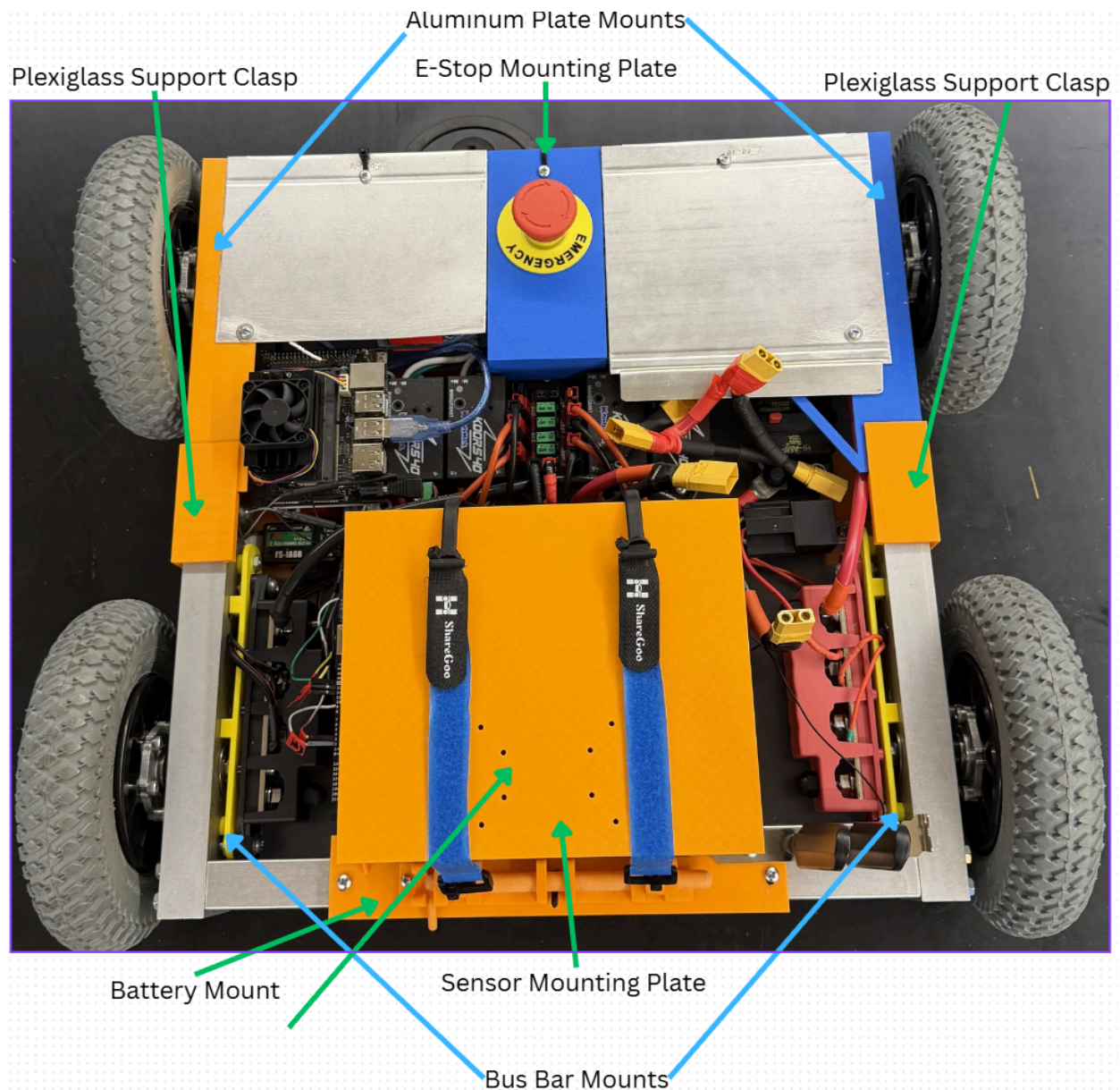


Figure 2: Overall layout (Jetson and motor controller/power distribution board mounts are blocked by all other parts but they are detailed below)

4.2.2. Iterative Mechanical Design Process

Detailed and accurate mechanical drawings were not available for the rover or for some of the components. Therefore almost every custom part's dimensions were found with direct

physical measurements using precise digital calipers and a ruler. These measurements were then plugged into Fusion 360 as parameters and used accordingly.

As a result of this the design process followed an iterative prototyping phase where the initial part would be modeled off of these measurements and then 3D printed. The printed part would then be installed and test fit and any errors would then be corrected in CAD before another version was printed. For larger parts such as the battery mount smaller test fitting parts were designed to confirm the dimensions were correct before printing the full part.

Every custom designed mount on the rover required at least one redesign during the iterative process and most required modifications after the print. Many of the redesigns were simple measurement adjustments however some, such as the battery and arduino mount required changes to mounting geometry and component orientation.

This process was extremely important for overall design quality as well because as parts were mounted and physically tested additional problems such as wire access, structural rigidity, component access and interference became apparent leading to more design tweaks and changes.

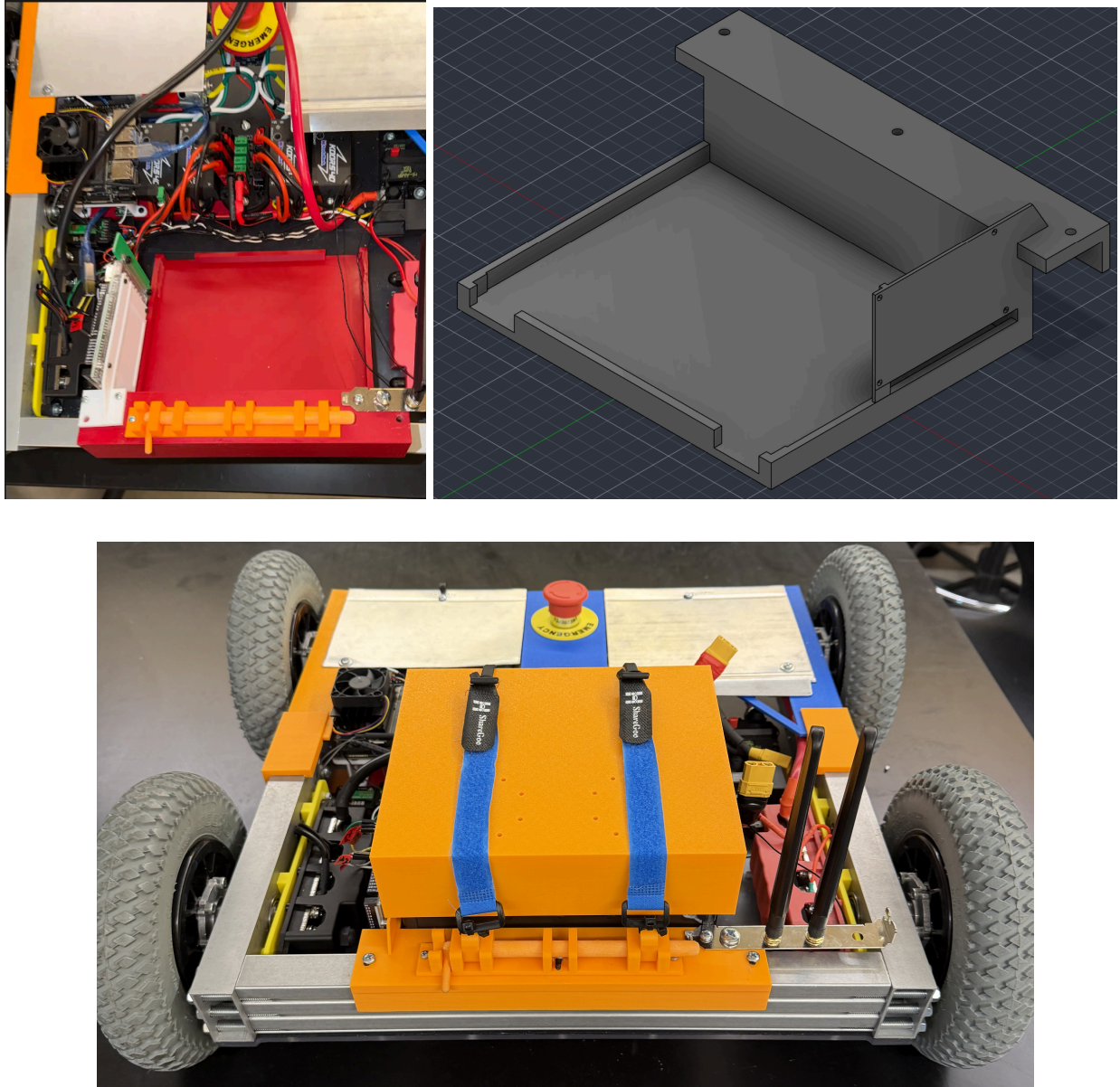
4.2.3. Battery and Arduino Mounting System

The largest mechanical challenge that came with designing the mechanical systems was securely mounting the large almost 12 pound battery with the limited space and weak plexiglass bottom that struggled to support its weight. The battery is by far the heaviest component of the system and its mount not only needs to keep it secure during acceleration and deceleration but it must also allow the battery to be removed for easier maintenance and charging.

The custom battery mount was designed to tightly match the battery's dimensions within a base keeping a lot of the weight off of the plexiglass bottom and instead transferring it to the

strong front metal beam of the chassis. In order to secure the battery from moving vertically out of its enclosure a custom sliding bolt mechanism was 3D printed and screwed onto the mount itself. Using two adjustable straps, the battery handle as well as the two keeps designed to interface with the bolt mechanism the battery is strapped down and secured (Shown in figure 5).

Initially the battery mount was not designed for the Arduino as the original plan was to only use the Jetson for all control. However once it was clear that the arduino was needed it became clear that because of the very limited space the arduino would have to be mounted sideways along the side of the battery. Initially this mount was separately printed and screwed into the battery mount but strength issues caused the redesign to incorporate the Arduino mount into the battery mount. Both versions can be seen in figures 3 and 4.



Figures 3a, 3b and 3c: Figure 3a shows the original battery mount and added arduino mount, 3b shows CAD redesign and 3c shows full implementation.

4.2.4.Main Electronics Mounting Plate

The second major mechanical component was designed to support the motor controllers, power distribution board, DC to DC converter and include mounting holes for the custom Jetson Nano mount. This was a result of the limited space available in the chassis and for ease of

integration. Having all these parts close together not only allowed for easier wiring but also made it so individual mounts would not have to be designed for each of the components.

The primary challenge of designing this mounting plate was deciding how the motor controllers and power distribution board would be attached directly above the plexiglass base. This is because once the plate is installed there is no clearance underneath to access mounting holes or place nuts. To solve this recessed nut locations were designed for all of the relevant mounting holes directly on the plate. These spaces hold the nuts in place and stop them from spinning so that the bolts can be screwed in from above after the plate has already been mounted using the predrilled plexiglass mounting holes.

Additionally the mounting plate includes two mounting holes for the DC to DC converter and another four for the separate raised Jetson nano mount. By combining these components onto a single mount the plexiglass plate does not need to be modified in any way and the layout remains compact.

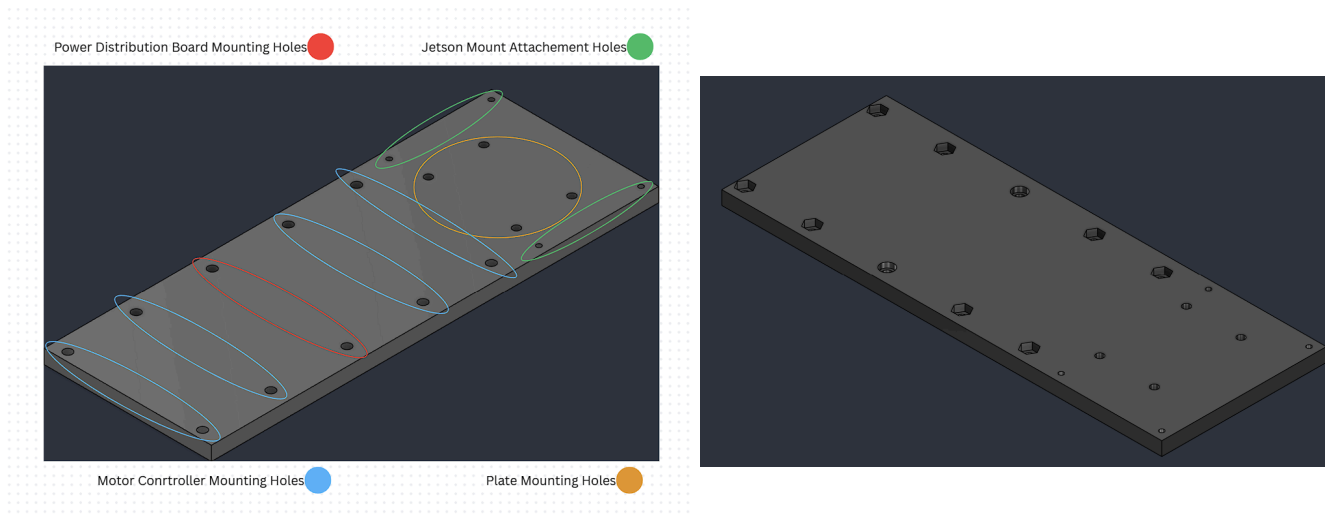


Figure 4a & 4b: 4a shows the top of the mounting plate CAD model and 4b shows the bottom.

4.2.5. Jetson Nano Raised Mount

The Jetson Nano required its own raised mount as mounting it directly to the main plate would cause extremely limited space to reach the USB A ports and significantly reduced airflow for cooling. As mentioned above the main plate was designed with four mounting holes specifically for the Jetson Mount. The Jetson mount itself was designed to allow space for the DC to DC converter mounted below as well as space for cable management later on. The mount was designed at a specific height to ensure all connectivity ports were accessible and so that the mount itself could be easily switched out later on in case the Jetson gets replaced.

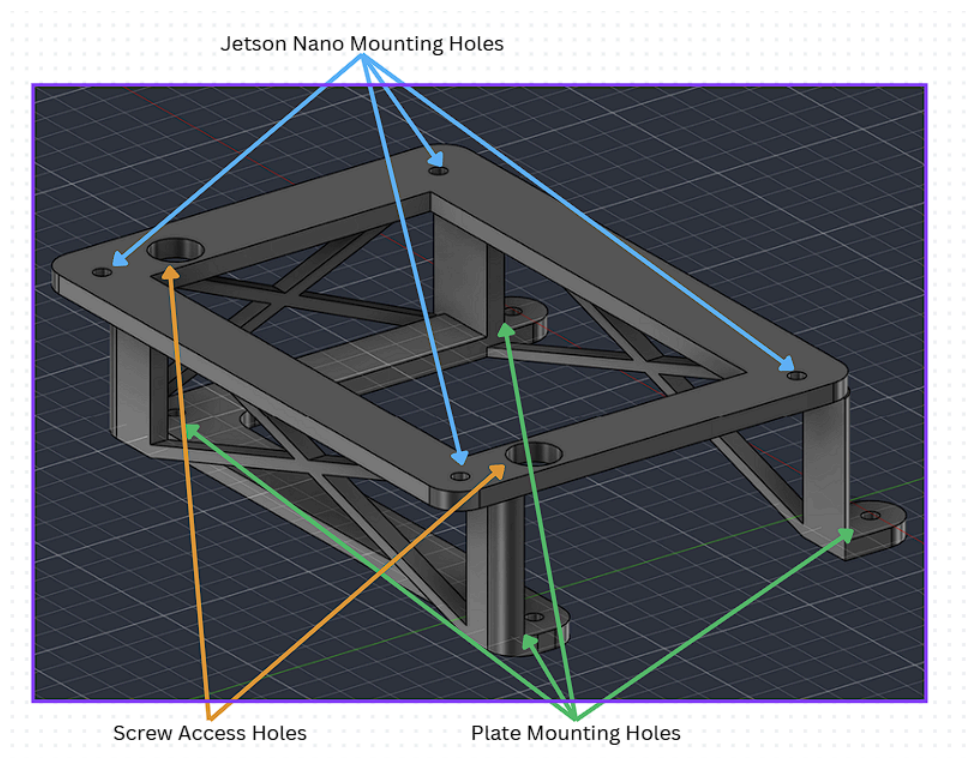


Figure 5: Jetson Nano raised mounting structure.

4.2.6. Bus Bar Mounts

The positive and negative bus bars used for system power distribution also required custom side mounting plates as once again drilling holes into the already weak plexiglass had to

be avoided. These mounts were designed to position the bus bars sideways up against the metal walls of the chassis and make their terminals easily accessible for wiring connections.

Like all of the other parts, many measurements of the chassis were taken to determine which of the existing mounting holes on the chassis were to be used and how the mounting holes on the part itself were to be placed. The mounting plate also features a raised gap in between the plate and the metal chassis to avoid making contact with the axis bolt that sticks out of the chassis on each side.

Mounting the bus bars next to the battery, near the 120A breaker and relay allowed for a convenient wiring layout. The extra space around these bus bars on the plexiglass also allows a significant amount more room for other bus bar terminals to be wired and used down the line.



Figure 6a & 6b: 6a shows the bus bar mount attached to the chassis holding the positive bus bar.

6b shows the CAD model.

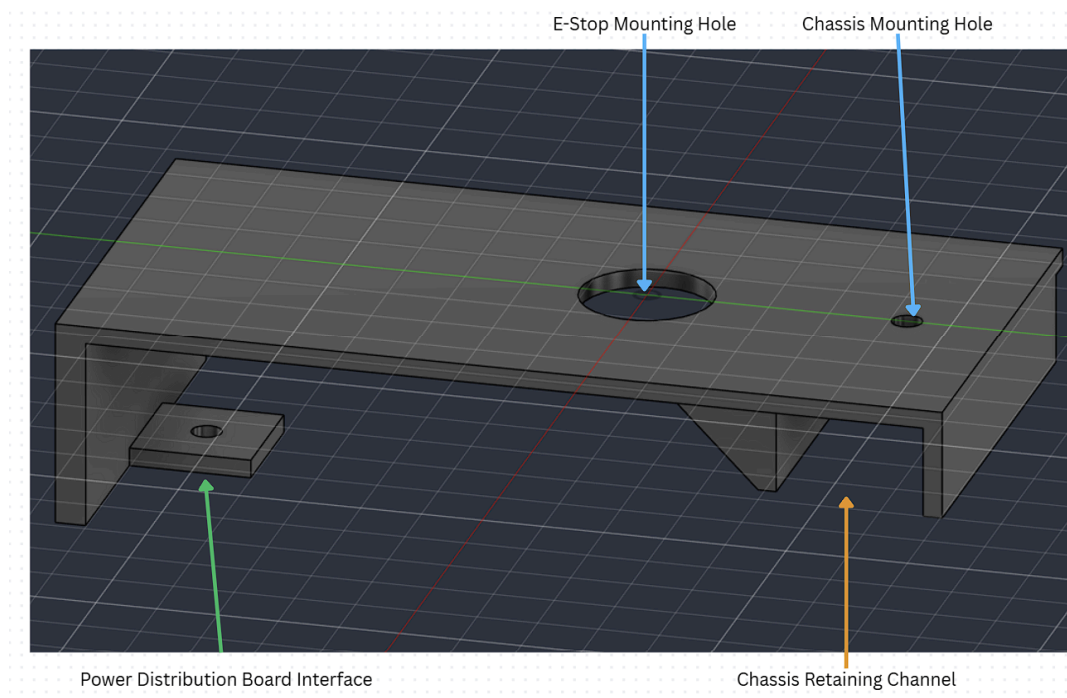
Note: Positive and negative bus bars both use identical mounts.

4.2.7.e-stop Mount

Mounting the e-stop button was an important consideration for the rover as it needs to be easily accessible and operable in case of a test failure. This is why it was mounted on the back of the rover in between the gearboxes and motors.

The mount was initially designed just to hold the e-stop, however the final design was a full plate that helps cover the motor wiring. Its inner support is designed to interface with the existing power distribution block mounting bolt in order to support the plate and hold it in place during use. The other end of the plate slots on top of the rear of the chassis using a retaining channel. It is then secured in place using the chassis mounting hole and power distribution board interface.

Note: More about the electrical side of the e-stop can be found in section 4.4.

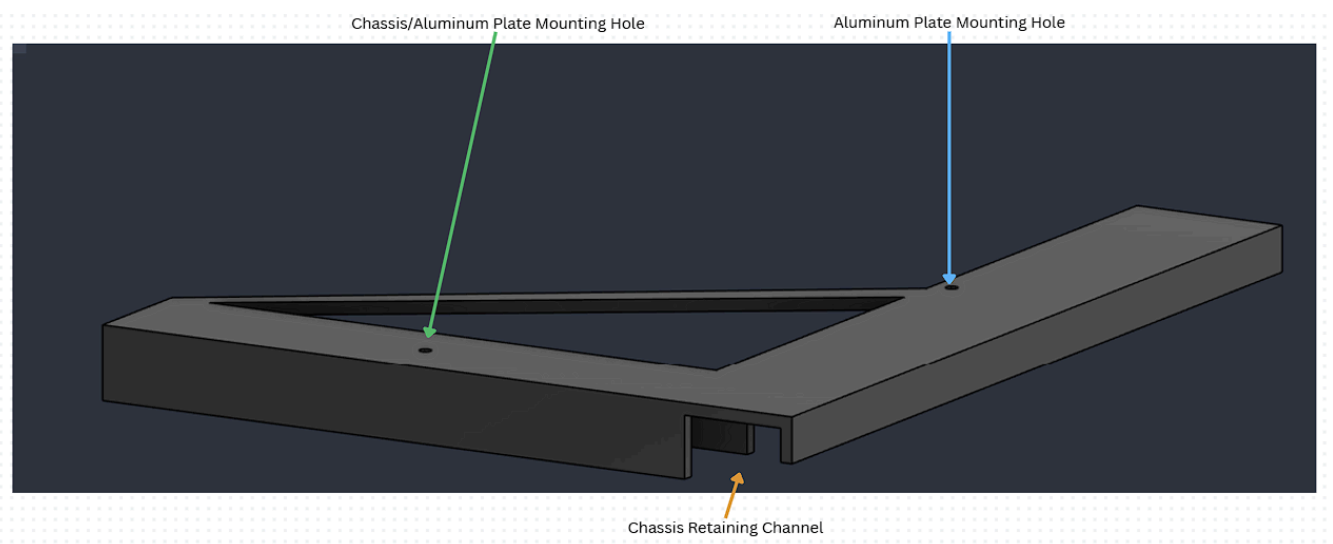


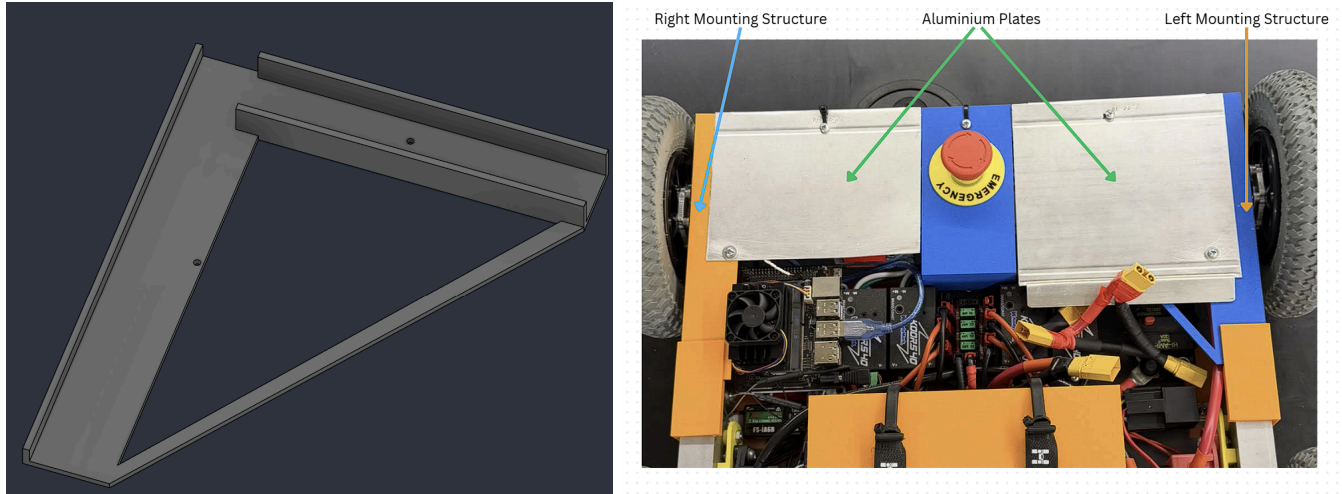
Figures 7a & 7b: 7a shows CAD model, 7b shows the mount installed in the rover with the e-stop button.

4.2.8.Wiring Covers and Drivetrain Protection

Mechanical mounts were designed to cover up the wiring to the motors, the motors themselves and the gearboxes. This was not only done for protection of the wires but also for operators pressing the e-stop button that is located directly between the motors and gearboxes. However because the area that these plates had to cover was large and the only structure to mount them on was the metal sides of the chassis it was determined that the best material to physically cover this area would be metal.

Specifically aluminum plates that were available in the lab would be used to cover the area. To do this custom mounting structures were designed for each side of the rover. These held the aluminum plates in place for seamless integration as can be seen in figure 8. The mounts for each side were nearly identical and like the e-stop mount both used the chassis retaining channel and were held in place through a chassis mounting hole.





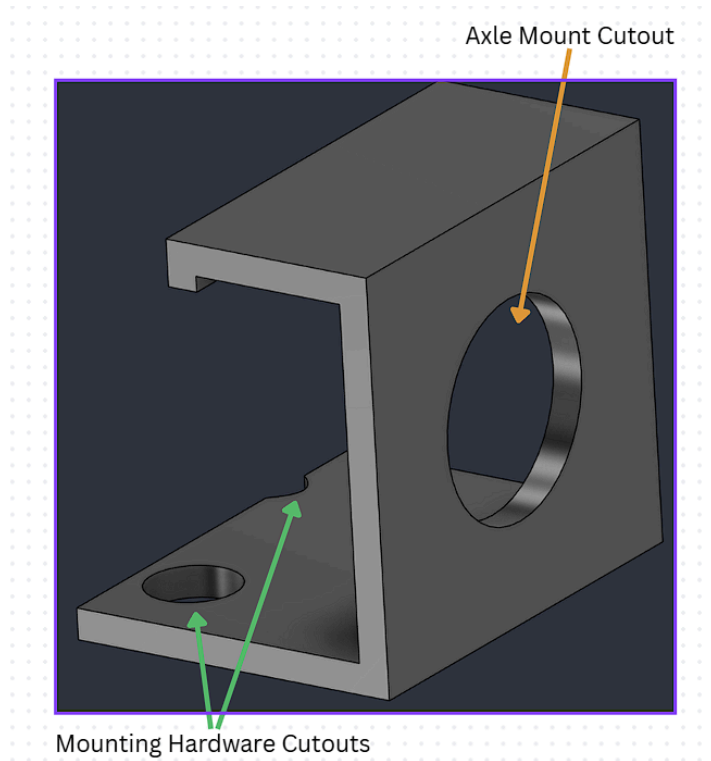
Figures 8a, 8b and 8c: 8a shows the top of the right side mounting plate CAD model, 8b shows the bottom of the left side mounting plate CAD model and 8c shows them both installed on the chassis with the aluminum plates mounted.

4.2.9. Plexiglass Structural Support

During the layout and fitting process of design it quickly became clear that the plexiglass bottom bent considerably even under light loads. Repeated flexing under load especially during operation could cause permanent damage to the plexiglass.

The solution to this were custom designed support clips that clamped the plexiglass close to the metal chassis significantly limiting any vertical movement under load by the plexiglass. These clips were designed around the existing chassis geometry in order to accommodate axles and are mounted without any external support so they can quickly be replaced or modified if needed.

Overall the supports were extremely valuable in eliminating the risk of the plexiglas breaking under the load of all the components.



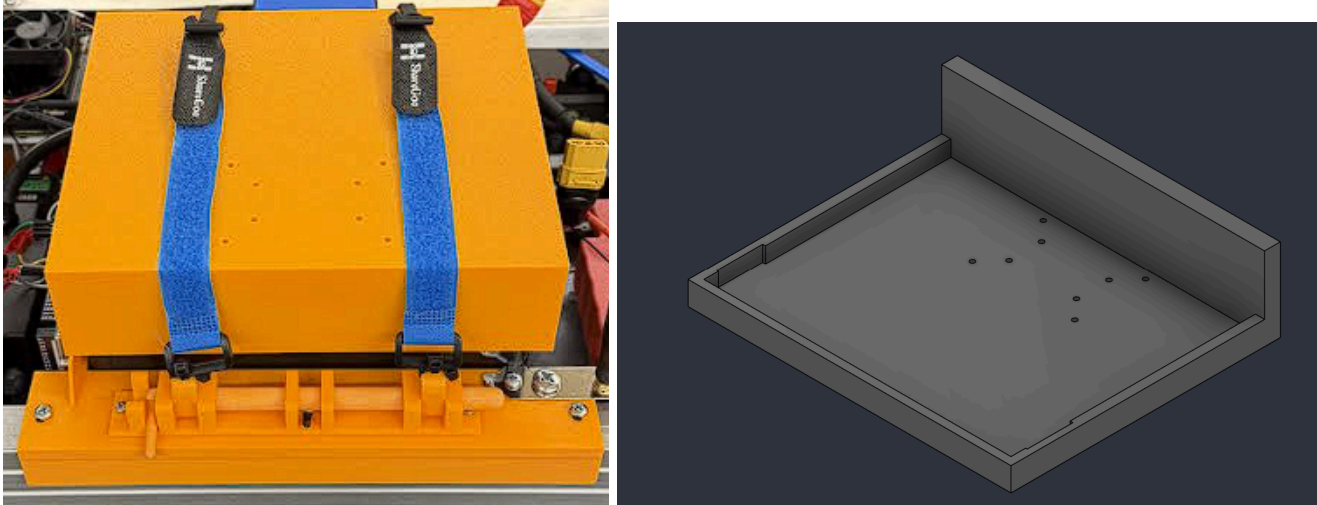
Figures 9a & 9b: 9a shows the left side clip CAD model and 9b shows it installed on the rover.

Note: The clips for each side are nearly identical; the right side did not require mounting hardware cutouts.

4.2.10.Modular Sensor Platform

A major requirement of the rover is the ability to add sensors later without rebuilding the primary mechanical layout. For this reason, a sensor mounting platform was designed with eight mounting holes allowing future users to design and attach custom sensor mounts to the rover. Since these sensors will need to be central, the mounting plate was designed to interface with the top of the battery eliminating horizontal movement while the battery straps hold it and the battery in place vertically.

The most important design goal for future sensor mounts is repeatable positioning. Sensors used for perception and localization will benefit from known orientation and height, so future brackets should use this fixed mounting reference.



Figures 10a & 10b: 10a shows the sensor mounting plate installed on the rover and 10b shows the bottom of the plate as a CAD model.

4.2.11. Iterative Prototyping and Final Integration

Overall the mechanical system consists of several independent mounting systems that work together to secure all electrical components and ensure structural integrity of the rover. The battery and arduino mount secures the heaviest component, makes space for the low level controller and allows for simple battery extraction. The central mounting plate for the power distribution board, motors, DC to DC converter and the raised Jetson Nano mount organizes the layout allowing excellent integration. Side mounts for the bus bars position them in ideal locations and orientations. Drivetrain cover mounts and the e-stop mount protect the motors, their wiring and the operator. Finally the structural plexiglass clips reduce flexing eliminating the possibility of permanent damage.

Using many individual mounts was an intentional design choice as it allows for easier modification and better upgradability in the future. This also means that the layout itself was chosen not just because the parts fit in the chassis. Wiring clearance, airflow, structural support,

safety, component accessibility and future modification were all considered throughout this iterative design process.

4.3.Motor Control Architecture

The drivetrain of the chassis requires independent variable speed and bidirectional control for both motor pairs. So the first stage of the design involved choosing the communication protocol that the motor controllers would use. The two options considered were Pulse Width Modulation (PWM) and Controller Area Network bus (CAN bus). PWM was selected because the requirements and precision of the rover is ultimately not enough to justify the added complexity of CAN bus. Although CAN provides features such as error detection, message prioritization, and network level communication, the rover's motor control requirements could easily be met using independent PWM signals. As a result each motor was paired with a Koors40 motor controller as they were an inexpensive well reviewed option that offered integrated PWM control. Each of these motor controllers are powered using the power distribution board and each have independent PWM connections. However it is important to note that once again each gearbox is driven by two motors and therefore that motor pair must receive the same PWM command from the Arduino.

4.3.1.Initial Jetson PWM Interface and Arduino Integration

The original motor control architecture was designed to use the Jetson Nano for both high level computing and direct PWM generation to the four Koors40 motor controllers. This would have simplified the system by eliminating the need for a separate low level controller. However, during initial testing it was determined that the PWM signal produced directly by the Jetson Nano was not reliably recognized by the motor controllers.

During early testing, the Jetson Nano was originally intended to generate the PWM control signals directly for the motor controllers. However, oscilloscope measurements showed that the Jetson output did not maintain a clean PWM waveform when connected to the motor controller input. The signal became significantly rounded and resembled a sinusoidal waveform rather than a sharp digital pulse. In contrast, the Arduino Mega produced a clean and stable PWM signal that was immediately accepted by the motor controllers. For this reason, the final design used the Jetson Nano for high level control and the Arduino Mega for low level PWM generation. This modification also created a clearer separation between high level computing and low level hardware control, which ultimately improved the modularity of the system.



Figure 11: Figure 11a shows the distorted PWM signal being sent from the Jetson and 11b shows the correct neutral PWM signal being sent from the Arduino.

4.4.e-stop Architecture

A physical e-stop button on the rover was a must have not only for testing ML software but also for basic functionality testing. Initially the plan was to implement an e-stop button that

cut power to the whole rover. However after proposing the plan to Matthew Olins it was made clear that troubleshooting and testing would be facilitated by the separation of control and powertrain systems allowing for the Jetson nano to remain operational even when power to the motors is cut. After further research and some deliberation Dr. Kalgaonkar it was determined that the rover power system should be split into three systems. The primary system would power the control systems, the secondary system would control a relay and the tertiary circuit powers the drivetrain. This was necessary as a secondary circuit is needed to separate the drivetrain and control circuits while the third circuit was needed because each motor can pull up to 120A at stall meaning most e-stop buttons would be unable to handle the immense power draw. This power draw would therefore be handled with a four contact 120A relay designed for automotive purposes. Implementing this requires a normally open (N/O) e-stop configuration paired with a normally closed relay (N/C). This circuit can be seen in figure 10 below.

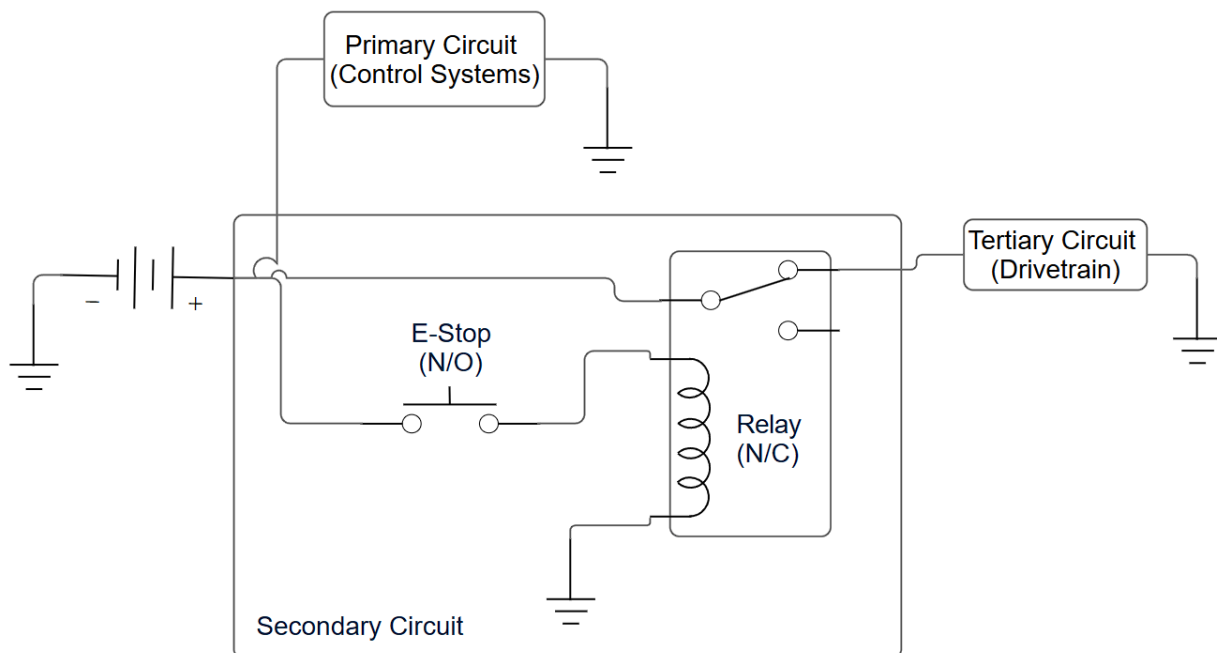


Figure 10: E-Stop Circuit Implementation

Note: During operation when the e-stop is pressed the tertiary circuit is connected, activating the coil in the relay which in turn opens the secondary circuit cutting power to only the drivetrain.

4.5.Electrical Power Architecture

Now that the main safety feature has been established the entire power distribution architecture has to be designed and implemented. The first step in this was choosing how the entire system would be powered.

4.5.1.Battery Selection and Main Circuit Protection

A 12 V rechargeable battery was selected to power the complete system while providing enough capacity for extended testing and enough discharge capability for the drivetrain. A 50 Ah LiFePO₄ battery with an internal battery management system (BMS) was selected. LiFePO₄ chemistry was preferred because it offers high cycle life, useful current capability, and a comparatively stable chemistry for a rechargeable high capacity testbed battery.

A 120 A AndyMark main circuit breaker was also added between the battery and the rover distribution system. The breaker provides a manual disconnect and overcurrent protection independent from the battery's internal BMS. The breaker and BMS should not be considered interchangeable because their trip thresholds and response times can differ. The 120 A breaker was also convenient because the Rover chassis already includes mounting provisions for this style of main breaker. The main reasoning for including this was again upgradeability as in the future different batteries without BMS protection can now be swapped in as long as they can handle the high current draw.

4.5.2.Drivetrain Power and Branch Protection

A dedicated fused power distribution board was used to supply the four motor controller branches. This approach keeps the drivetrain wiring organized and provides individual circuit protection so a fault in one motor branch does not rely only on the main breaker or battery protection.

Each RedLine motor is capable of drawing approximately 120 A at stall, while each Koors40 controller is rated for 40 A. Four 30A snap action breakers were therefore used as conservative branch protection for the four motor controller circuits. These breakers limit sustained branch current and help protect wiring and controllers, but the 30 A ratings should not be interpreted as a precise 120 A instantaneous system current limit because thermal breakers have time current trip behavior and brief transients can exceed the 30A value.

4.6.Main Circuit Power Distribution

Since it was determined that control systems should still be powered after an e-stop press, the power distribution board used for the drivetrain could not be used to power the control systems. Since these circuits have to be separate a second method of power distribution had to be used. One option would have been to use another power distribution board but this was expensive and unnecessary because the breakers would not be needed. As a result online research was conducted to find modular solutions to to easily wire a DC circuit. Ultimately it was determined that power distribution bus bars would be a perfect solution. These bars allow for ring terminal connections so soldering or splicing of wires is not needed, they can easily handle high current draw and any future additions of technology can be added on the extra terminal points. Joinworld Power Distribution Block Bus Bars were chosen as the positive and negative power distribution hubs for the battery. Each comes with four M8 stud terminals and three M4 screw terminals leaving three terminals unused for future upgrades. Figure 11

demonstrates the final architecture used in the design showing how the system is split into three subcircuits as the e-stop architecture mentions above.

Powering the control electronics is a straightforward process once the system is separated into subcircuits. The Jetson Nano uses a barrel jack which requires a stable and reliable 5V 4A minimum input. To supply this the Pololu 5V, 5A Step-Down Voltage Regulator was chosen for its high performance, efficiency and small profile. The Arduino Mega can be powered with a USB B connection. Since there are many extra USB A ports available on the Jetson and because the Jetson Nano needs to communicate with the Arduino Mega the USB interface was the best solution for power and communication. This control system also involves the wireless receiver which was chosen as the FS-iA6B because it was already on hand and easy to work with. This receiver uses a 5V input for power which was done using the 5V out and ground pins on the Arduino Mega seen in figure 11.

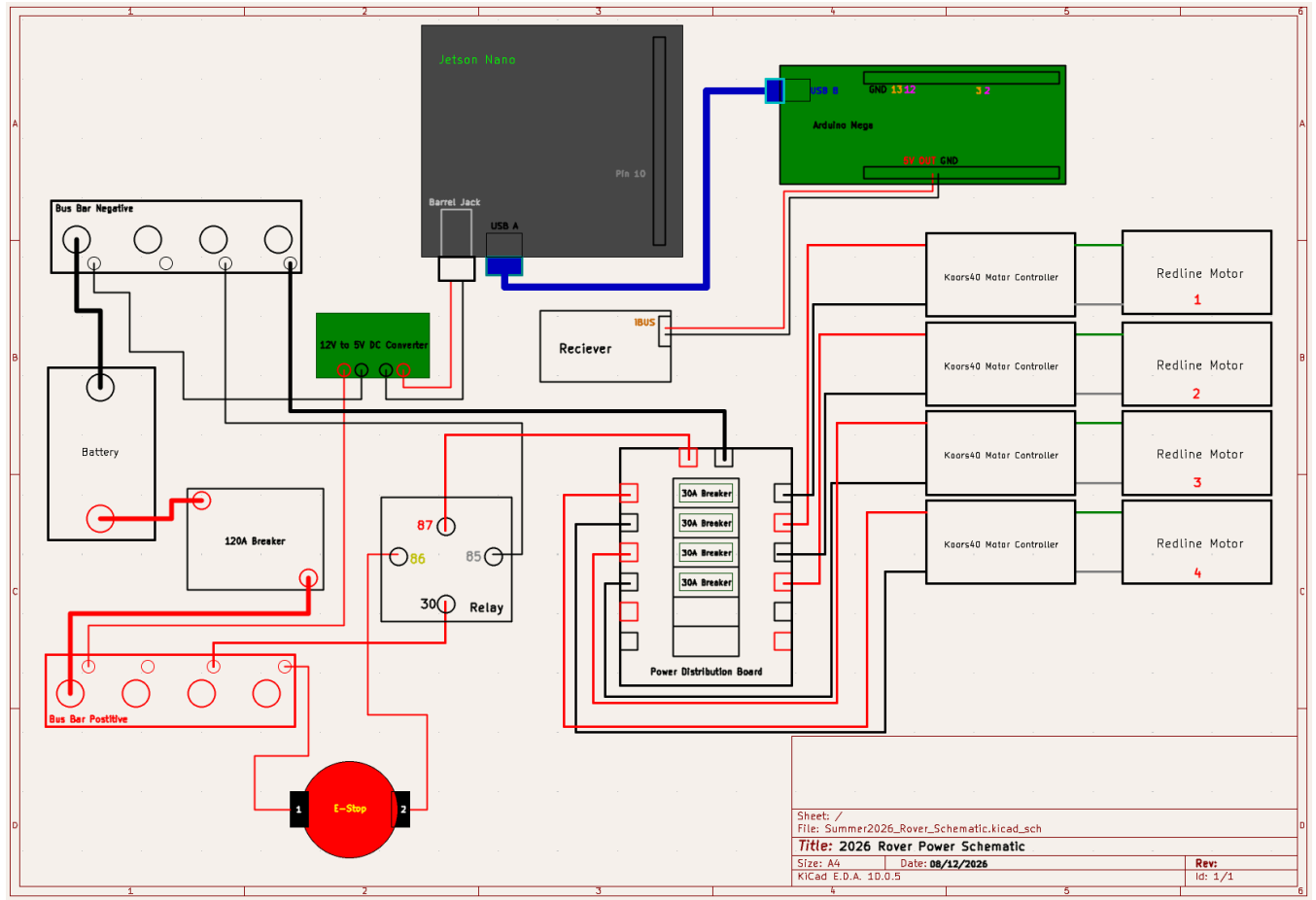


Figure 11: Main Power Circuit Architecture

4.7.Computing Architecture

The computing architecture is intentionally divided between a Jetson Nano and an Arduino Mega. The Jetson is responsible for high level tasks that will likely be computationally intensive, including wireless command processing, future perception, machine learning inference, autonomous control logic, logging, and command generation. The Arduino Mega is simply tasked with receiving left and right side commands then generating the four motor controller control pulses.

This separation reduces the amount of timing-sensitive hardware control performed by Linux on the Jetson and prevents future changes to autonomous software from requiring changes to the motor-controller interface. It also creates a clear upgrade path. A future computer board can replace the Jetson as long as it preserves the same simple serial command interface to the Arduino.

4.8. Wireless Manual Control Architecture

Manual control is provided using a FlySky FS-i6S transmitter paired with an FS-iA6B 2.4 GHz receiver. The receiver supports iBUS output in addition to conventional individual PWM channels, allowing multiple controller channels to be transmitted over one digital serial connection. The Jetson's program reads this channel data and uses selected channels for steering, throttle, an enable switch, and a three position speed selector.

The two position switch named SWA on the controller acts as a software enable. When the rover is not enabled, the Jetson forces the drivetrain command to neutral no matter what commands are being sent to the controller at the time. This switch is to act as the main drivetrain disabler during ML testing with the e-stop button on the rover itself being last resort. The three position switch selects low, medium, or high command limits. These modes correspond to 20%, 30%, and 40% of the full $\pm 500 \mu\text{s}$ command range around $1500 \mu\text{s}$ neutral command. The high mode therefore matches the 40% output limit enforced again by the Arduino, while the two lower modes provide more precise control for testing.

Mode	Command percentage	Maximum delta	Nominal output range
Low	20%	$\pm 100 \mu\text{s}$	1400–1600 μs
Medium	30%	$\pm 150 \mu\text{s}$	1350–1650 μs
High	40%	$\pm 200 \mu\text{s}$	1300–1700 μs

Table 2: Manual speed mode command limits.

4.9.Communication Architecture

The rover uses three primary communication interfaces. The controller communicates wirelessly with the receiver at 2.4 GHz. The receiver sends iBUS channel data to the Jetson over a serial interface. The Jetson then sends compact left- and right-side commands to the Arduino through USB serial at 115200 baud. Finally, the Arduino generates the four motor-controller control pulses.

Devices	Interface	Purpose
FS-i6S Controller to FS-iA6B Receiver	2.4GHz	Operator commands
FS-iA6B to Jetson Nano	iBUS	Channel data
Jetson Nano to Arduino Mega	USB Serial at 115200 Baud	Left and right side drive commands.
Arduino Mega to Koors40 Motor Controllers	PWM	Motor speed and direction

Table 3: Primary rover communication interfaces.

The Jetson to Arduino command was made simple for debugging purposes. Each command is sent as two integer pulse widths separated by a comma and terminated by a newline, for example “1500,1500\n”. This makes serial testing and debugging straightforward while keeping parsing on the Arduino minimal.

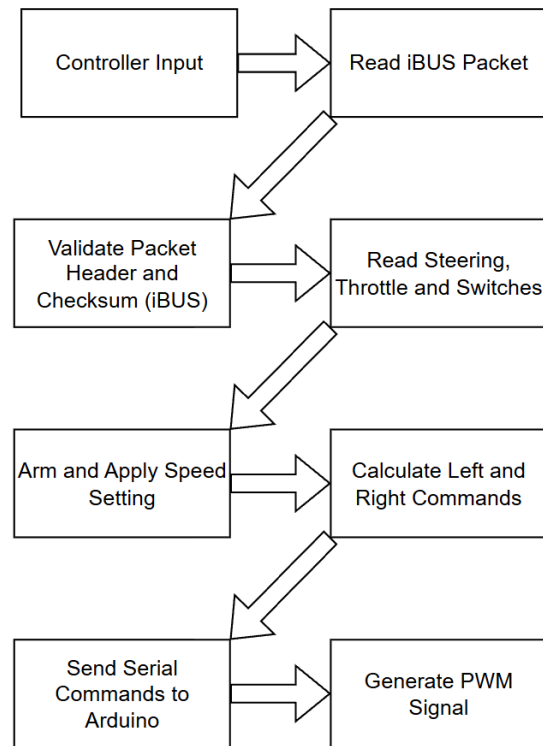


Figure 12: Software control flow diagram.

5. Software and Control Implementation

5.1. Software Architecture Overview

The software architecture follows the same high/low level separation used in the hardware architecture. The Jetson runs the primary Python control program and is responsible for interpreting operator commands and deciding what left and right side power should be requested. The Arduino runs the low level motor control program in C/C++ and is responsible for interpreting those requests, limiting them to the accepted range, and continuously generating the four motor-controller control signals.

This separation also creates two independent software safety layers. If the wireless receiver stops providing valid input, the Jetson forces neutral. If the Jetson stops communicating with the Arduino for any reason, the Arduino independently forces neutral after its own timeout.

5.2.Jetson Nano Control Program

5.2.1.iBUS Input Processing

The Jetson receives the controller data using the FlySky iBUS serial protocol. A standard iBUS control frame contains 32 bytes: a two byte header, fourteen little endian channel values, and a two byte checksum. The program identifies the expected header, verifies the checksum, and then decodes the channel values before any control commands are generated. Typical channel values are approximately 1000 to 2000 with 1500 representing the neutral position for all channels.

Only validated packets are used to update the rover command state. Invalid headers, incomplete packets, or checksum failures are ignored. This prevents corrupted serial data from directly becoming a motor command.

5.2.2.Throttle and Steering

Raw steering and throttle channels are limited to the expected 1000-2000 input range and then normalized to a floating point range from -1.0 to +1.0. A small deadzone around the center prevents small joystick offsets or receiver noise from causing unintended motion when the controls are released. Channel inversion flags allow steering or throttle direction to be reversed in software without changing the rest of the control logic.

5.2.3.Skid Steer Command Generation

The normalized throttle and steering commands are converted into left and right side commands using differential drive mixing. Essentially $\text{left} = \text{throttle} + \text{turn}$ and $\text{right} = \text{throttle} -$

turn. If either command exceeds the allowed normalized magnitude, both commands are scaled by the maximum absolute magnitude to preserve the input ratio and keep each output within ± 1.0 . This method allows forward motion, reverse motion, gradual turns, and point turns using the same two joystick inputs.

5.2.4.Speed Modes

Three selectable speed modes were implemented using a three position switch (SWB) on the wireless controller. The final Jetson program limits the commanded PWM deviation to ± 100 μs around the 1500 μs neutral point. Low, medium, and high modes apply scaling factors of 40%, 70%, and 100% to this allowable command range, respectively. This results in maximum PWM deviations of approximately ± 40 μs , ± 70 μs , and ± 100 μs for the three modes.

5.2.5.Arming and Safety Logic

A two position controller switch was implemented as the software arming control. To prevent unintended motion during startup, the rover cannot arm simply because the switch is already in the enabled position. The program must first detect the switch in the OFF position. Once this condition has been satisfied, the rover will only arm when the switch is moved to ON while both the throttle and steering inputs are within a 10% neutral deadzone. This prevents an immediate motor command from being applied if either joystick is displaced when the rover is armed.

The Jetson also tracks the time of the most recent valid iBUS packet. If the receiver data is not updated within the configured 0.25 s timeout, the software sends neutral commands instead of continuing to use the last valid joystick position. This is important because a disconnection should not allow the rover to continue moving. Once again in the case of all software failsafe failures the physical e-stop button will always shut the drivetrain down.

5.3.Jetson Nano to Arduino Mega Serial Communication

The Jetson transmits commands to the Arduino every 0.05 s. Each message contains the requested left and right pulse widths in microseconds using the format “leftPWM,rightPWM\n” for simple debugging. It is important to note that the receiver and the arduino are both connected to the Jetson but both use different serial connections. The final deployed Jetson program therefore requires the correct device path for each one.

5.4.Arduino Mega Motor Control Program

5.4.1.Serial Parsing

The Arduino receives serial data into a fixed 32 byte character buffer. A newline marks the end of each command. The program locates the comma separating the two values, converts the left and right strings to integers, and rejects commands that are not within the accepted 1000–2000 μs input range. Oversized messages are also rejected and the buffer is reset.

5.4.2.PWM Generation

Four Servo library objects are attached to Arduino pins 2, 3, 12, and 13. The program uses writeMicroseconds() so the command can be expressed directly in the same pulse width units used throughout the high level software and Koors40 documentation. Neutral is defined as 1500 μs .

5.4.3.Motor Direction and Inversion

The four motors do not all share the same physical orientation and wiring direction, so the program has boolean inversion tags applied before each pulse is written. The command is calculated by subtracting the requested pulse from double the neutral value of 1500 μs . This means, for example, that 1600 μs becomes 1400 μs for an inverted motor while neutral remains

unchanged. This was done in the Arduino program as the Jetson program was intended to be used mainly for the left and right side directions.

5.4.4. Output Limits

Although the Koors40 accepts approximately 1000-2000 μs commands, the Arduino applies an additional software power limit. With a neutral value of 1500 μs , a nominal full delta of 500 μs , and `POWER_LIMIT` set to 40, the final Arduino output range is constrained to 1300-1700 μs . This means even if the Jetson sends a larger but otherwise valid value, the Arduino limits the command before it reaches the motor controller. This was done to reduce the chance that any high level software errors would trip the circuit breakers.

5.5. Software Failsafes

As mentioned previously the rover uses several software and hardware layers rather than relying on any single safety mechanism. The controller enable switch prevents intentional commands while disarmed, the Jetson receiver timeout prevents stale wireless commands from remaining active, the Arduino serial timeout forces neutral if communication from the Jetson stops, and the Arduino output clamp limits the maximum pulse range. The hardware drivetrain stop provides an additional independent method of removing motor power.

Safety layer	Location	Trigger	Response
Remote enable/arming switch	Jetson	Enable switch off	All motor outputs set to neutral
Wireless input timeout	Jetson	No valid iBUS packet for 0.25 s	All motor outputs set to neutral
Command timeout	Arduino	No valid Jetson command for 0.50 s	All motor outputs set to neutral
Output clamp	Arduino	Command exceeds +/-200 μ s configured limit	Command constrained to 1300-1700 μ s
Physical drivetrain stop	Power system	Operator actuates e-stop button	High current drivetrain path opened by relay

Table 4: Layered control and safety mechanisms.

6. System Integration and Testing

6.1. Integration Approach

System testing was performed progressively rather than energizing the complete rover immediately after assembly. Individual subsystems were first checked independently and then combined as confidence in the wiring and software increased. This method reduced the chance that an error in one subsystem would damage another component or make debugging difficult.

The general integration sequence was: verify low voltage control power, verify the Jetson and Arduino serial connection, verify neutral motor controller signals with drivetrain power controlled, test one side of the drivetrain at low command, verify motor direction and inversion, test manual steering and throttle, confirm the arming and speed switches, verify both software timeout behaviors, and finally verify the physical drivetrain stop during integrated operation. All of these functions are videoed and documented on the rover's Github page linked below.

6.2. Power and Safety Validation

Before drivetrain testing, the control power branch was verified so the Jetson, Arduino, and receiver could operate without motor motion. The power distribution board and relay controlled drivetrain branch were then tested to confirm that upon an e-stop activation drivetrain power was cut while the control electronics remained on. This directly validated the primary reason for separating the control and drivetrain power architectures. Also part of this phase was determining power stability to all electronics, especially the 5V rail powering the Jetson Nano and therefore all control systems.

6.3.Communication and Wireless Control Validation

The Jetson to Arduino serial connection was first tested using known neutral and directional commands. The Arduino was verified to parse comma separated pulse widths and return the motors to neutral if valid commands stopped arriving. The wireless receiver was then integrated and individual controller channels were observed to confirm correct mapping of steering, throttle, arming, and speed mode inputs.

The receiver timeout was tested by interrupting wireless command updates and confirming that the high level program commanded neutral rather than retaining the last joystick value. Separately, the Arduino timeout was tested by interrupting Jetson command transmission and confirming that the low level controller independently returned all motor outputs to neutral.

6.4.Motor and Drivetrain Validation

Motor testing started at limited command values to confirm direction and identify any motors that required inversion. The left side motors were then confirmed to respond to the same logical left command and each right side motor to the same logical right command. Forward, reverse, gradual turning, and point turn behavior were tested after all four motors were enabled.

The 40% Arduino output limit kept motor controller commands between 1300 and 1700 μ s during these tests. The three controller speed modes were then verified to increase speeds for the larger command ranges while still being constrained to the Arduino limits.

6.5.Manual Control and Final Testing

The final manual control test combined the complete power, wireless, software, serial, and drivetrain architecture. The rover was driven using the FlySky transmitter with the arming switch enabled and was tested in low, medium, and high speed modes. The arming switch was then disabled to verify that joystick movement no longer drove the motors. The physical e-stop and both communication failsafes were also exercised during the integrated testing process.

These tests demonstrated the intended functional behavior of the completed testbed: the rover can be manually controlled, speed limited, disarmed remotely, stopped through loss of signal logic, stopped through loss of Jetson to Arduino communication, and physically disabled at the drivetrain while leaving the control computer powered.

7.Results

The primary result of the project is a complete rover testbed with integrated mechanical mounting, power distribution, high level computing, low level motor control, wireless manual control, and layered shutdown behavior. The platform separates high level software development from low level motor control allowing future software to generate the same left/right command interface without needing to understand the details of individual motor controller wiring or motor inversion.

The mechanical design also achieved the intended modularity. Rather than permanently modifying the chassis for every component, removable 3D-printed mounts were used to organize

the battery, Arduino, Jetson, motor controllers, bus bars, e-stop button, covers, and structural support. The iterative process required more redesign than hoped but it allowed the final layout to account for real connector access, wiring, cooling, and chassis tolerances.

The power architecture provides a useful debugging advantage by keeping the Jetson and Arduino powered when the drivetrain is disabled. This is especially valuable for autonomous development because logs and software state can remain available after a test is stopped.

The software architecture is similarly layered. The Jetson performs command interpretation and drive mixing, but the Arduino independently validates and limits the final pulse widths. The two timeout layers mean that a failure at either the wireless input stage or the Jetson to Arduino stage results in neutral motor commands. These layers do not eliminate every possible failure mode, but they significantly reduce dependence on a single software process.

8.Limitations and Future Work

While this testing successfully demonstrated proper functionality of the rover with all design requirements met, more rigorous testing will likely need to take place before full autonomous implementation. For starters electronics temperature measurements, and extended reliability testing were not conducted and should likely be tested before autonomous operation.

Second autonomous operation requires additional hardware not yet implemented on the rover. Any sensors added will require software development, integration and testing before any ML algorithms can be used. However because the rover was designed around this these sensors can be added without changing the low level motor control.

Third is the Jetson should run a script on startup that automatically runs the final Jetson high level control software to avoid having to set it up with peripherals and a monitor each time

the system reboots. The Jetson also should be added to the school network to allow remote connection to devices on the same network.

9. Conclusion

This project successfully designed and integrated a four wheel drive skid steer rover intended to serve as a safe, reliable, and repeatable platform for future autonomous control development. Starting from an AndyMark Rover chassis and Jetson Nano, the project added a custom mechanical layout, 3D-printed mounting hardware, a dedicated power distribution system, drivetrain branch protection, a hardware drivetrain stop, a low level Arduino motor controller, and wireless manual control.

The final control architecture separates high level computation from low level motor signal generation. The Jetson processes iBUS receiver data, implements arming and speed mode logic, generates skid steer commands, and communicates with the Arduino. The Arduino independently validates the commands, applies output limits and motor inversion, and generates the four Koors40 control signals. Multiple software timeouts and the independent hardware drivetrain disconnect provide layered responses to communication or control failures.

Most importantly, the completed rover provides a flexible foundation rather than a single purpose vehicle. Its modular mechanical, electrical, and software architecture allows future researchers to add sensors, replace computing hardware, develop autonomous algorithms, and perform sim to real testing without redesigning the complete platform. The project therefore achieves its main objective of creating the mechatronic and embedded control infrastructure needed for continued autonomous rover development.

References

- 1) AndyMark, “Rover,” product page and Rover 8 in. Pneumatic chassis specifications, accessed Aug. 31, 2026. <https://andymark.com/products/rover>
- 2) AndyMark, “Koors40 Brushed Speed Controller (am-5600),” user manual and specifications, accessed Aug. 31, 2026.
<https://docs.andymark.com/motor-controllers/koors40-brushed-speed-controller>
- 3) AndyMark, “AndyMark 775 RedLine Motor,” product specifications, accessed Aug. 31, 2026.
<https://www.andymark.com/products/andymark-775a-redline-motor>
- 4) NVIDIA, “Power supply considerations for Jetson Nano Developer Kit,” NVIDIA Developer Forums / Jetson Nano power guidance, accessed Aug. 31, 2026.
<https://forums.developer.nvidia.com/t/power-supply-considerations-for-jetson-nano-developer-kit/71637>
- 5) Pololu, “5V, 5A Step-Down Voltage Regulator D24V50F5,” specifications, accessed Aug. 31, 2026. <https://www.pololu.com/product/2851>
- 6) FlySky, “FS-i6S Digital Proportional Radio Control System User Manual,” receiver and iBUS interface documentation.
https://sdr-files.s3.us-east-2.amazonaws.com/product_info/FS-i6S%20User%20manual.pdf
- 7) E. Aljalbout *et al.*, “The Reality Gap in Robotics: Challenges, Solutions, and Best Practices,” *Annual Review of Control, Robotics, and Autonomous Systems*, vol. 9, pp. 403–432, 2026, doi: 10.1146/annurev-control-031924-100130.

Appendix: Project Links

The following links provide access to the final rover software and supporting demonstration videos.

1. GitHub Repository - Final Rover Code

Link: <https://github.com/rfrobins/Autonomous-Rover-Testbed>

2. Youtube Video - Driving Demonstration

Link: <https://www.youtube.com/watch?v=0dPNv8tmsvw>

3. Youtube Video - Emergency Stop Functionality

Link: <https://www.youtube.com/watch?v=sDpVlpVIPuo>

4. Youtube Video - Other Features

Link: <https://www.youtube.com/watch?v=tUtS9Z2b58U>